EmbryoGuard Pro: Distributed Project Report

This document contains the report divided equally among the 5 team members, with each receiving a balanced portion of the content (approximately 2-3 sections each, similar length and importance).

Person 1 (Project Lead & Firmware Engineer)

EmbryoGuard Pro: IoT-Based Smart Embryo Incubator System
Embedded Systems Project Report
Instructor: Dr. Safa Elaskary

Assigned Sections:

- Section 1: Introduction
- Section 2: System Overview & Architecture
- Section 10: FreeRTOS Task Architecture

1. Introduction

1.1 Project Background

Embryo incubation is a critical process in poultry farming that requires precise environmental control to ensure optimal hatch rates. Traditional incubators often lack real-time monitoring, remote access capabilities, and intelligent automation. The EmbryoGuard Pro system addresses these limitations by leveraging modern IoT technologies, embedded systems, and web-based dashboards to create a comprehensive, intelligent incubator management platform.

1.2 Project Objectives

- Design and implement a dual-node IoT incubator monitoring and control system
- Achieve precise temperature regulation using hysteresis-based control with ±0.5°C accuracy
- Support multiple egg species (Chicken, Duck, Quail, Goose) with dynamic threshold adjustment
- Provide real-time web-based dashboard with WebSocket communication for instant updates

- Implement FreeRTOS-based multitasking for concurrent sensor reading and actuator control
- Enable both automatic and manual control modes with safety override mechanisms
- Log all sensor data to a persistent SQLite database for historical analysis
- Deliver a mobile-accessible interface with QR code connectivity

1.3 Scope

This project encompasses the complete design cycle from hardware selection and circuit design, through firmware development on the ESP32 microcontroller using FreeRTOS, to a full-stack Flask + WebSocket web dashboard with real-time charting, alert systems, and actuator controls.

2. System Overview & Architecture

2.1 High-Level Architecture

The EmbryoGuard Pro system follows a dual-node architecture running on a single ESP32 microcontroller, utilizing both CPU cores via FreeRTOS. The system communicates with a Flask-based web dashboard over Wi-Fi using HTTP REST APIs and WebSocket for real-time updates.

The architecture consists of three main layers:

- **Hardware Layer:** Sensors (DHT22, MQ-135, LDR) and actuators (relay-controlled fan/lamp, servo motor)
- **Firmware Layer:** ESP32 running FreeRTOS with two main tasks pinned to separate cores
- **Application Layer:** Flask web server with SocketIO, SQLite database, and responsive HTML/CSS/JS dashboard

2.2 Data Flow

Node 1 (Core 1) reads temperature/humidity from DHT22, regulates the environment via relays and servo, and POSTs sensor data to the Flask server every 3 seconds. Node 2 (Core 0) independently monitors a second DHT22, MQ-135 gas sensor, and LDR light sensor, also POSTing data every 3 seconds. The Flask server stores data in SQLite, evaluates alerts, and broadcasts updates to all connected browser clients via WebSocket. Commands from the dashboard are queued and polled by Node 1 every 3 seconds.

10. FreeRTOS Task Architecture

Task	Core	Stack Size	Priority	Function
node1Task	Core 1	10240 bytes	1	Climate control, sensor read, HTTP POST, command poll
node2Task	Core 0	10240 bytes	1	Environmental monitoring, sensor read, HTTP POST
servoTask	Core 1	4096 bytes	2	Smooth servo sweeping with attach/detach management

Two FreeRTOS mutexes protect shared resources: **http_mutex** serializes HTTP client access across cores, and **serial_mutex** synchronizes Serial.printf calls to prevent interleaved output.

14. References (Partial)

- Espressif Systems. "ESP32 Technical Reference Manual." 2024.
- FreeRTOS. "FreeRTOS Real-Time Kernel Documentation." freertos.org, 2024.

Person 2 (Dashboard Developer)

Instructor: Dr. Safa Elaskary

Assigned Sections:

- Section 4: Software Architecture
- Section 7: Communication Protocol
- Section 11: Database & Data Logging

4. Software Architecture

4.1 Firmware Stack

- **Language:** C/C++ (Arduino framework)
- **RTOS:** FreeRTOS (built into ESP-IDF/Arduino core)
- **Libraries:** DHT.h, ESP32Servo.h, WiFi.h, HTTPClient.h, ArduinoJson.h
- **Watchdog:** 10-second hardware watchdog timer (esp_task_wdt)
- **Concurrency:** Mutex-protected HTTP and Serial access via FreeRTOS semaphores

4.2 Dashboard Stack

- **Backend:** Python 3, Flask 3.0.0, Flask-SocketIO 5.3.6
- **Frontend:** HTML5, CSS3 (custom design system), vanilla JavaScript
- **Real-time:** WebSocket via Socket.IO (threading async mode)
- **Database:** SQLite3 for persistent sensor logging
- **Charting:** Chart.js for real-time temperature/humidity graphs
- **QR Code:** qrcodejs library for mobile connectivity
- **Typography:** Google Fonts (Inter, weights 300-800)

7. Communication Protocol

7.1 ESP32 → Server (HTTP POST)

Node 1 JSON payload (every 3 seconds):

```
{
  "node": 1,
  "temperature": 37.5,
  "humidity": 55.0,
  "fan": false,
  "lamp": "OFF",
  "servo_angle": 90,
  "servo_turns": 5,
  "incubation_day": 3,
  "sensor_ok": true,
  "cycle": 42,
  "mode": "AUTO",
  "egg_type": "chicken"
}
```

Node 2 JSON payload (every 3 seconds):

```
{
  "node": 2,
  "temperature": 37.2,
  "humidity": 52.0,
  "mq135_raw": 350,
  "mq135_volts": 0.28,
  "mq135_dout": 1,
  "light_pct": 15,
  "sensor_ok": true,
  "cycle": 40
}
```

7.2 Server → ESP32 (Command Queue)
Commands are queued in the Flask server and retrieved by Node 1 via polling GET requests. Supported commands: mode (AUTO/MANUAL), fan (true/false), lamp (OFF/FULL), servo (turn), egg_type (chicken/duck/quail/goose), sweep (ms), dwell (ms).

7.3 Server ↔ Browser (WebSocket)
The browser establishes a Socket.IO connection on page load. Events include: "sensor_update" (server → client), "control" (client → server for actuator commands), "command_ack" (server → client confirmation), "egg_type_update", and "settings_update".

11. Database & Data Logging

All sensor data is persistently logged to an SQLite database (incubator.db). The sensor_log table stores timestamped readings with node ID, temperature, humidity, and additional metadata serialized as JSON in the "extra" column. Up to 500 data points are kept in memory for real-time charting, with the database providing full historical records.

Column	Type	Description
id	INTEGER PK	Auto-incrementing record ID
ts	TEXT	ISO 8601 timestamp
node	INTEGER	Node ID (1 or 2)
temperature	REAL	Temperature in °C
humidity	REAL	Relative humidity %
extra	TEXT (JSON)	Additional sensor data (fan, lamp, servo, gas, light, etc.)

14. References (Partial)

- Flask Documentation. "Flask Web Development, One Drop at a Time." flask.palletsprojects.com.
- Socket.IO. "Socket.IO Documentation." socket.io/docs, 2024.

Person 3 (Frontend & UI Engineer)

Instructor: Dr. Safa Elaskary

Assigned Sections:

- Section 6: Dashboard & Web Interface
- Section 8: Multi-Species Egg Profiles

6. Dashboard & Web Interface

6.1 Architecture

The dashboard is built with Flask and served on port 8080. It uses Flask-SocketIO for bidirectional real-time communication. The ESP32 nodes POST data via HTTP REST API, and the server broadcasts updates to all browser clients via WebSocket events.

6.2 Features

- Real-time circular gauge visualizations for temperature and humidity (Node 1)
- Sensor cards with color-coded status badges for Node 2 (temp, humidity, ammonia, light)
- Interactive actuator controls: Mode toggle (AUTO/MANUAL), Fan, Lamp, Egg Turner
- Egg type selector with 4 species and dynamic threshold updates
- Incubation progress tracker with day counter and phase indicator
- Live Chart.js graphs for temperature and humidity history
- Multi-level alert system with danger/warning indicators
- QR code generator for instant mobile device connectivity
- Responsive CSS design with glassmorphism aesthetic and dark accents

6.3 API Endpoints

Endpoint	Method	Description
/	GET	Serve main dashboard HTML page
/api/data	POST	Receive sensor data from ESP32 nodes
/api/commands/<nid>	GET	ESP32 polls for queued commands
/api/state	GET	Browser fetches full system state on page load
/api/settings	POST	Update automatic mode threshold settings

Endpoint	Method	Description
/api/egg-type	POST	Change egg type and apply profile thresholds
/api/server-info	GET	Return server IP/port for QR code generation

8. Multi-Species Egg Profiles

EmbryoGuard Pro supports four egg species, each with scientifically calibrated temperature thresholds and incubation periods. Profile selection is synchronized between dashboard and firmware.

Species	Target Temp (°C)	Lamp ON Below (°C)	Fan ON Above (°C)	Incubation Days
Chicken 🐔	37.5	36.5	38.5	21
Duck 🦆	37.7	37.0	39.0	28
Quail 🐦	37.8	37.5	39.5	18
Goose 🦢	37.6	36.8	38.8	25

14. References (Partial)

- Chart.js. "Chart.js Open Source HTML5 Charts." chartjs.org, 2024.
- ArduinoJson. "ArduinoJson v7 Documentation." arduinojson.org, 2024.

Person 4 (Node 2 Firmware & Sensor Integration)

Instructor: Dr. Safa Elaskary

Assigned Sections:

- Section 3: Hardware Design
- Section 5: Firmware Implementation (Node 2 focus)

3. Hardware Design

3.1 Microcontroller – ESP32

The LILYGO TTGO T-Display ESP32 was selected as the main microcontroller. It features a dual-core Xtensa LX6 processor running at 240 MHz, 520 KB SRAM, integrated Wi-Fi 802.11 b/g/n, and Bluetooth 4.2. The dual-core architecture is essential for running two independent sensor nodes concurrently without interference.

3.2 Sensor Suite

Sensor	Model	GPIO Pin(s)	Function	Specifications
Temperature/Humidity #1	DHT22	GPIO 21	Climate control feedback (Node 1)	±0.5°C, ±2% RH
Temperature/Humidity #2	DHT22	GPIO 22	Environmental monitoring (Node 2)	±0.5°C, ±2% RH
Gas/Ammonia Sensor	MQ-135	GPIO 32 (A), GPIO 17 (D)	Air quality monitoring	Analog + Digital out
Light Sensor	LDR	GPIO 33	Lid open detection / light level	12-bit ADC, 0-100%

3.3 Actuators

Actuator	Type	GPIO Pin	Control Method	Function
Cooling Fan	2-Channel Relay (IN1)	GPIO 27	Active-HIGH digital	Temperature reduction
Heating Lamp	2-Channel Relay (IN2)	GPIO 26	Active-HIGH digital	Temperature increase
Egg Turner	SG90 Servo Motor	GPIO 13	50 Hz PWM (500-2400µs)	Periodic egg rotation

3.4 Pin Mapping Summary

Node	GPIO	Direction	Component
Node 1	21	Input	DHT22 Data
Node 1	27	Output	Relay – Fan (Active HIGH)
Node 1	26	Output	Relay – Lamp (Active HIGH)
Node 1	13	Output	Servo PWM
Node 2	22	Input	DHT22 Data
Node 2	32	Analog In	MQ-135 AOUT

Node	GPIO	Direction	Component
Node 2	17	Digital In	MQ-135 DOUT
Node 2	33	Analog In	LDR Analog

5. Firmware Implementation

5.4 Node 2 – Sensor Monitor
Node 2 runs on Core 0 and provides independent environmental monitoring. It reads a second DHT22, MQ-135 gas sensor (with multi-sample ADC averaging), and LDR light sensor. All readings are posted to the dashboard for display and alerting. Node 2 does not control any actuators.

5.5 Manual Mode & Safety (Shared)
The system supports MANUAL mode where the operator can directly control fan, lamp, and servo from the dashboard. Key safety features include:

- MANUAL mode auto-reverts to AUTO after 15 minutes of inactivity
- Danger zone temperatures ALWAYS override manual commands
- Sensor failure causes all actuators to shut down (fail-safe)
- Dashboard buttons are locked (grayed out) when AUTO mode is active

14. References (Partial)

- Aosong Electronics. "DHT22 Digital Temperature and Humidity Sensor Datasheet." 2020.
- Zhengzhou Winsen Electronics. "MQ-135 Gas Sensor Technical Data." 2019.

Person 5 (Hardware & Testing Engineer)

Instructor: Dr. Safa Elaskary

Assigned Sections:

- Section 9: Safety & Alert System
- Section 12: Testing & Validation
- Section 13: Conclusion & Future Work

9. Safety & Alert System

9.1 Temperature Alerts

Condition	Level	Action
T ≥ 39.5°C	DANGER (Lethal)	Emergency fan ON, lamp OFF – overrides all modes
T ≤ 35.0°C	DANGER (Arrest)	Emergency lamp ON, fan OFF – overrides all modes
T > temp_hot	WARNING	Dashboard warning indicator
T < temp_cold	WARNING	Dashboard warning indicator

9.2 Humidity Alerts

Condition	Level	Risk
H ≥ 75%	DANGER	Mold risk
H ≤ 38%	DANGER	Desiccation risk
H > 65%	WARNING	Humidity high
H < 45%	WARNING	Humidity low

9.3 Environmental Alerts

Sensor	Warning Threshold	Danger Threshold
MQ-135 (Ammonia)	ADC ≥ 800	ADC ≥ 1200 or DOUT = LOW
LDR (Light)	≥ 60%	≥ 85% (lid open?)

9.4 System Health

- Node offline detection: 15-second heartbeat timeout
- DHT22 sensor failure tracking with fail count / total count
- Watchdog timer: 10-second hardware WDT resets ESP32 on firmware hang
- WiFi auto-reconnect with persistent disabled to prevent flash wear

12. Testing & Validation

12.1 Hardware Testing

- Relay self-test sequence on boot: Fan ON 600ms → OFF, Lamp ON 600ms → OFF
- Servo initialization: attach, write center position, wait 500ms, detach
- ADC calibration: 12-bit resolution with 11dB attenuation for full 0-3.3V range
- Multi-sample averaging (5 samples) for MQ-135 and LDR readings to reduce noise

12.2 Communication Testing

- HTTP POST timeout set to 2500ms to prevent blocking on network issues
- WiFi connection retry: 20 attempts × 400ms, graceful fallback to standalone mode
- WebSocket connection with CORS wildcard for cross-origin mobile access
- Command queue clear-on-read to prevent duplicate command execution

12.3 Safety Testing

- Danger zone override verified: manual commands ignored at extreme temperatures
- Manual mode 15-minute auto-revert timeout confirmed
- Sensor failure fail-safe: all actuators OFF when DHT22 returns NaN
- Watchdog timer reset confirmed on firmware hang simulation

13. Conclusion & Future Work

13.1 Conclusion

The EmbryoGuard Pro system successfully demonstrates a complete IoT-based embryo incubator solution integrating embedded systems, real-time communication, and web technologies. The dual-core FreeRTOS architecture enables concurrent sensor monitoring and actuator control without interference. The hysteresis-based temperature regulation provides stable climate control within ±0.5°C of the target, and the multi-species egg profile system makes the platform versatile for different poultry types. The responsive web dashboard provides operators with comprehensive real-time monitoring, alerting, and control capabilities accessible from any device on the local network.

13.2 Future Enhancements

- Cloud integration (AWS IoT / Firebase) for remote monitoring beyond the local network
- Machine learning-based predictive analytics for hatch rate optimization
- Camera module integration for visual egg candling and embryo development tracking
- GSM/SMS alert notifications for critical events when Wi-Fi is unavailable
- OTA (Over-The-Air) firmware update capability via the dashboard
- Multi-incubator support with a centralized management dashboard
- Battery backup with UPS monitoring and graceful shutdown
- Historical data export (CSV/PDF) for farm record keeping

14. References (Partial)

- All references from sections above are shared collectively.

Summary of Equal Division

Person	Role	Sections Assigned
Person 1	Project Lead & Firmware Engineer	1 (Introduction), 2 (Architecture), 10 (FreeRTOS)
Person 2	Dashboard Developer	4 (Software Stack), 7 (Communication), 11 (Database)

Person	Role	Sections Assigned
Person 3	Frontend & UI Engineer	6 (Dashboard UI), 8 (Egg Profiles)
Person 4	Node 2 Firmware & Sensor Integration	3 (Hardware), 5 (Firmware - Node 2)
Person 5	Hardware & Testing Engineer	9 (Safety & Alerts), 12 (Testing), 13 (Conclusion)

Each person has **2-3 sections** of roughly equal length and technical importance. The Table of Contents, Title, Instructor line, and References are shared or split as shown.